```
#without value
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>3D Missile Launcher</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            overflow: hidden;
            background: #0a0a0f;
            font-family: 'Segoe UI', system-ui, sans-serif;
        }
        #canvas-container {
            width: 100vw;
            height: 100vh;
        }
        #ui-panel {
            position: fixed;
            top: 20px;
            left: 20px;
            background: linear-gradient(135deg, rgba(20, 25, 30, 0.95), rgba(10,
15, 20, 0.98));
            border: 1px solid rgba(80, 200, 120, 0.3);
            border-radius: 12px;
            padding: 20px;
            color: #e0e0e0;
            z-index: 100;
            box-shadow: 0 8px 32px rgba(0, 0, 0, 0.5), inset 0 1px 0
rgba(255,255,255,0.05);
        }
        #ui-panel h1 {
            font-size: 18px;
            margin-bottom: 15px;
            color: #50c878;
            text-transform: uppercase;
            letter-spacing: 2px;
            font-weight: 600;
        }
        .btn {
            display: block;
            width: 100%;
            padding: 12px 20px;
            margin: 8px 0;
            background: linear-gradient(135deg, #1a2f1a, #0d1f0d);
            border: 1px solid #50c878;
            color: #50c878;
            font-size: 13px;
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 1px;
            cursor: pointer;
            border-radius: 6px;
            transition: all 0.3s ease;
        }
        .btn:hover {
            background: linear-gradient(135deg, #50c878, #3da85c);
            color: #0a0f0a;
            box-shadow: 0 0 20px rgba(80, 200, 120, 0.4);
```

```
            transform: translateY(-2px);
        }
        .btn:active {
            transform: translateY(0);
        }
        .btn.danger {
            border-color: #ff6b35;
            color: #ff6b35;
            background: linear-gradient(135deg, #2f1a1a, #1f0d0d);
        }
        .btn.danger:hover {
            background: linear-gradient(135deg, #ff6b35, #cc5429);
            color: #0a0f0a;
            box-shadow: 0 0 20px rgba(255, 107, 53, 0.4);
        }
        .info {
            margin-top: 15px;
            padding-top: 15px;
            border-top: 1px solid rgba(80, 200, 120, 0.2);
            font-size: 11px;
            color: #888;
            line-height: 1.6;
        }
        .stats {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 10px;
            margin-top: 15px;
        }
        .stat-item {
            background: rgba(0,0,0,0.3);
            padding: 10px;
            border-radius: 6px;
            text-align: center;
        }
        .stat-value {
            font-size: 20px;
            font-weight: bold;
            color: #50c878;
        }
        .stat-label {
            font-size: 10px;
            color: #666;
            text-transform: uppercase;
        }
        #loading {
            position: fixed;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            color: #50c878;
            font-size: 24px;
            z-index: 1000;
        }
    </style>
</head>
<body>
    <div id="canvas-container"></div>
    <div id="ui-panel">
        <h1>Missile Launcher</h1>
        <button class="btn danger" id="launchBtn">Launch Missile</button>
        <button class="btn" id="rotateTurretBtn">Rotate Turret</button>
        <button class="btn" id="resetBtn">Reset Position</button>
        <button class="btn" id="autoRotateBtn">Auto Rotate: ON</button>
```

```
        <div class="stats">
            <div class="stat-item">
                <div class="stat-value" id="missileCount">8</div>
                <div class="stat-label">Missiles</div>
            </div>
            <div class="stat-item">
                <div class="stat-value" id="launchCount">0</div>
                <div class="stat-label">Launched</div>
            </div>
        </div>
        <div class="info">
            Drag to rotate view<br>
            Scroll to zoom<br>
            Click vehicle parts to highlight
        </div>
    </div>

    <script type="importmap">
    {
        "imports": {
            "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
            "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
        }
    }
    </script>

    <script type="module">
        import * as THREE from 'three';
        import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

        // Scene setup
        const container = document.getElementById('canvas-container');
        const scene = new THREE.Scene();

        // Gradient background
        const bgCanvas = document.createElement('canvas');
        bgCanvas.width = 2;
        bgCanvas.height = 512;
        const bgCtx = bgCanvas.getContext('2d');
        const gradient = bgCtx.createLinearGradient(0, 0, 0, 512);
        gradient.addColorStop(0, '#0d1a0d');
        gradient.addColorStop(0.5, '#0a1210');
        gradient.addColorStop(1, '#050a08');
        bgCtx.fillStyle = gradient;
        bgCtx.fillRect(0, 0, 2, 512);
        const bgTexture = new THREE.CanvasTexture(bgCanvas);
        scene.background = bgTexture;

        // Camera
        const camera = new THREE.PerspectiveCamera(60, window.innerWidth /
window.innerHeight, 0.1, 1000);
        camera.position.set(12, 8, 12);

        // Renderer
        const renderer = new THREE.WebGLRenderer({ antialias: true });
        renderer.setSize(window.innerWidth, window.innerHeight);
        renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
        renderer.shadowMap.enabled = true;
        renderer.shadowMap.type = THREE.PCFSoftShadowMap;
        container.appendChild(renderer.domElement);

        // Controls
        const controls = new OrbitControls(camera, renderer.domElement);
        controls.enableDamping = true;
```

```
controls.dampingFactor = 0.05;
controls.autoRotate = true;
controls.autoRotateSpeed = 0.5;
controls.maxPolarAngle = Math.PI / 2.1;
controls.minDistance = 8;
controls.maxDistance = 30;

// Lighting
const ambientLight = new THREE.AmbientLight(0x404050, 0.5);
scene.add(ambientLight);

const mainLight = new THREE.DirectionalLight(0xfff5e0, 1.2);
mainLight.position.set(10, 20, 10);
mainLight.castShadow = true;
mainLight.shadow.mapSize.width = 2048;
mainLight.shadow.mapSize.height = 2048;
mainLight.shadow.camera.near = 0.5;
mainLight.shadow.camera.far = 50;
mainLight.shadow.camera.left = -20;
mainLight.shadow.camera.right = 20;
mainLight.shadow.camera.top = 20;
mainLight.shadow.camera.bottom = -20;
scene.add(mainLight);

const fillLight = new THREE.DirectionalLight(0x50c878, 0.3);
fillLight.position.set(-10, 5, -10);
scene.add(fillLight);

const rimLight = new THREE.DirectionalLight(0xff6b35, 0.2);
rimLight.position.set(0, 10, -15);
scene.add(rimLight);

// Materials
const materials = {
    body: new THREE.MeshStandardMaterial({
        color: 0x2d3d2d,
        roughness: 0.7,
        metalness: 0.3
    }),
    dark: new THREE.MeshStandardMaterial({
        color: 0x1a251a,
        roughness: 0.8,
        metalness: 0.2
    }),
    metal: new THREE.MeshStandardMaterial({
        color: 0x4a5a4a,
        roughness: 0.4,
        metalness: 0.7
    }),
    missile: new THREE.MeshStandardMaterial({
        color: 0x3a4a3a,
        roughness: 0.5,
        metalness: 0.6
    }),
    missileTip: new THREE.MeshStandardMaterial({
        color: 0xff6b35,
        roughness: 0.3,
        metalness: 0.5,
        emissive: 0xff6b35,
        emissiveIntensity: 0.2
    }),
    glass: new THREE.MeshStandardMaterial({
        color: 0x88ccaa,
        roughness: 0.1,
```

```
            metalness: 0.9,
            transparent: true,
            opacity: 0.6
        }),
        wheel: new THREE.MeshStandardMaterial({
            color: 0x1a1a1a,
            roughness: 0.9,
            metalness: 0.1
        }),
        highlight: new THREE.MeshStandardMaterial({
            color: 0x50c878,
            roughness: 0.3,
            metalness: 0.5,
            emissive: 0x50c878,
            emissiveIntensity: 0.5
        })
    };

    // Missile Launcher Group
    const missileLauncher = new THREE.Group();
    scene.add(missileLauncher);

    // Chassis
    const chassisGroup = new THREE.Group();
    missileLauncher.add(chassisGroup);

    // Main body
    const bodyGeom = new THREE.BoxGeometry(6, 1.5, 3);
    const body = new THREE.Mesh(bodyGeom, materials.body);
    body.position.y = 1.2;
    body.castShadow = true;
    body.receiveShadow = true;
    body.userData = { part: 'body' };
    chassisGroup.add(body);

    // Front cabin
    const cabinGeom = new THREE.BoxGeometry(2, 1.8, 2.6);
    const cabin = new THREE.Mesh(cabinGeom, materials.body);
    cabin.position.set(2.5, 2.3, 0);
    cabin.castShadow = true;
    cabin.userData = { part: 'cabin' };
    chassisGroup.add(cabin);

    // Cabin windows
    const windowGeom = new THREE.BoxGeometry(0.1, 0.8, 2);
    const windowFront = new THREE.Mesh(windowGeom, materials.glass);
    windowFront.position.set(3.51, 2.5, 0);
    chassisGroup.add(windowFront);

    const windowSide1 = new THREE.Mesh(new THREE.BoxGeometry(1, 0.6, 0.1),
materials.glass);
    windowSide1.position.set(2.8, 2.7, 1.31);
    chassisGroup.add(windowSide1);

    const windowSide2 = windowSide1.clone();
    windowSide2.position.z = -1.31;
    chassisGroup.add(windowSide2);

    // Rear section
    const rearGeom = new THREE.BoxGeometry(2.5, 2.2, 2.8);
    const rear = new THREE.Mesh(rearGeom, materials.dark);
    rear.position.set(-1.8, 2.5, 0);
    rear.castShadow = true;
    rear.userData = { part: 'rear' };
```

```
chassisGroup.add(rear);

// Turret base
const turretBaseGroup = new THREE.Group();
turretBaseGroup.position.set(-1.5, 3.6, 0);
chassisGroup.add(turretBaseGroup);

const turretBaseGeom = new THREE.CylinderGeometry(1.2, 1.4, 0.6, 8);
const turretBase = new THREE.Mesh(turretBaseGeom, materials.metal);
turretBase.castShadow = true;
turretBase.userData = { part: 'turret' };
turretBaseGroup.add(turretBase);

// Rotating turret
const turretGroup = new THREE.Group();
turretBaseGroup.add(turretGroup);

const turretGeom = new THREE.BoxGeometry(2.5, 0.8, 2);
const turret = new THREE.Mesh(turretGeom, materials.metal);
turret.position.y = 0.7;
turret.castShadow = true;
turretGroup.add(turret);

// Missile pods
const missilePodGroup = new THREE.Group();
missilePodGroup.position.set(-0.8, 1.3, 0);
missilePodGroup.rotation.z = Math.PI * 0.1;
turretGroup.add(missilePodGroup);

// Create missile pods
function createMissilePod(zOffset) {
    const podGroup = new THREE.Group();
    podGroup.position.z = zOffset;

    const podFrameGeom = new THREE.BoxGeometry(2.8, 1.4, 0.6);
    const podFrame = new THREE.Mesh(podFrameGeom, materials.dark);
    podFrame.castShadow = true;
    podGroup.add(podFrame);

    return podGroup;
}

const leftPod = createMissilePod(0.7);
const rightPod = createMissilePod(-0.7);
missilePodGroup.add(leftPod);
missilePodGroup.add(rightPod);

// Create missiles
const missiles = [];
const launchedMissiles = [];
let missileCount = 8;
let launchCount = 0;

function createMissile(zPos, pod) {
    const missileGroup = new THREE.Group();
    missileGroup.position.set(0.5, 0, zPos);

    const bodyGeom = new THREE.CylinderGeometry(0.12, 0.12, 1.2, 8);
    const body = new THREE.Mesh(bodyGeom, materials.missile);
    body.rotation.z = Math.PI / 2;
    body.castShadow = true;
    missileGroup.add(body);

    const tipGeom = new THREE.ConeGeometry(0.12, 0.3, 8);
```

```
    const tip = new THREE.Mesh(tipGeom, materials.missileTip);
    tip.rotation.z = -Math.PI / 2;
    tip.position.x = 0.75;
    missileGroup.add(tip);

    const finsGeom = new THREE.BoxGeometry(0.2, 0.3, 0.02);
    for (let i = 0; i < 4; i++) {
        const fin = new THREE.Mesh(finsGeom, materials.missile);
        fin.position.set(-0.5, 0, 0);
        fin.rotation.x = (Math.PI / 2) * i;
        fin.position.z = Math.cos((Math.PI / 2) * i) * 0.13;
        fin.position.y = Math.sin((Math.PI / 2) * i) * 0.13;
        missileGroup.add(fin);
    }

    missileGroup.userData = { launched: false, speed: 0, trail: null };
    pod.add(missileGroup);
    missiles.push(missileGroup);
    return missileGroup;
}

// Add missiles to pods
createMissile(0.15, leftPod);
createMissile(-0.15, leftPod);
createMissile(0.15, rightPod);
createMissile(-0.15, rightPod);

// Wheels
function createWheel(x, z) {
    const wheelGroup = new THREE.Group();
    wheelGroup.position.set(x, 0.5, z);

    const wheelGeom = new THREE.CylinderGeometry(0.5, 0.5, 0.4, 16);
    const wheel = new THREE.Mesh(wheelGeom, materials.wheel);
    wheel.rotation.x = Math.PI / 2;
    wheel.castShadow = true;
    wheelGroup.add(wheel);

    const hubGeom = new THREE.CylinderGeometry(0.25, 0.25, 0.45, 8);
    const hub = new THREE.Mesh(hubGeom, materials.metal);
    hub.rotation.x = Math.PI / 2;
    wheelGroup.add(hub);

    return wheelGroup;
}

// Add wheels
const wheelPositions = [
    [2, 1.7], [2, -1.7],
    [0.5, 1.7], [0.5, -1.7],
    [-1, 1.7], [-1, -1.7],
    [-2.5, 1.7], [-2.5, -1.7]
];

wheelPositions.forEach(pos => {
    const wheel = createWheel(pos[0], pos[1]);
    chassisGroup.add(wheel);
});

// Track guards
const guardGeom = new THREE.BoxGeometry(6.5, 0.15, 0.5);
const guardL = new THREE.Mesh(guardGeom, materials.dark);
guardL.position.set(0, 1, 1.8);
chassisGroup.add(guardL);
```

```
const guardR = guardL.clone();
guardR.position.z = -1.8;
chassisGroup.add(guardR);

// Radar antenna
const radarGroup = new THREE.Group();
radarGroup.position.set(3, 3.5, 0);
chassisGroup.add(radarGroup);

const antennaBase = new THREE.Mesh(
    new THREE.CylinderGeometry(0.1, 0.15, 0.5, 6),
    materials.metal
);
radarGroup.add(antennaBase);

const antennaArm = new THREE.Mesh(
    new THREE.BoxGeometry(0.08, 0.6, 0.08),
    materials.metal
);
antennaArm.position.y = 0.55;
radarGroup.add(antennaArm);

const radarDish = new THREE.Mesh(
    new THREE.SphereGeometry(0.3, 8, 4, 0, Math.PI * 2, 0, Math.PI / 2),
    materials.metal
);
radarDish.position.y = 0.85;
radarDish.rotation.x = Math.PI / 4;
radarGroup.add(radarDish);

// Details - exhaust pipes
const exhaustGeom = new THREE.CylinderGeometry(0.1, 0.12, 1, 8);
const exhaust1 = new THREE.Mesh(exhaustGeom, materials.metal);
exhaust1.rotation.z = Math.PI / 2;
exhaust1.position.set(-2.8, 2, 1.3);
chassisGroup.add(exhaust1);

const exhaust2 = exhaust1.clone();
exhaust2.position.z = -1.3;
chassisGroup.add(exhaust2);

// Ground plane
const groundGeom = new THREE.PlaneGeometry(50, 50);
const groundMat = new THREE.MeshStandardMaterial({
    color: 0x1a201a,
    roughness: 0.9
});
const ground = new THREE.Mesh(groundGeom, groundMat);
ground.rotation.x = -Math.PI / 2;
ground.receiveShadow = true;
scene.add(ground);

// Grid
const gridHelper = new THREE.GridHelper(50, 50, 0x2a3a2a, 0x1a251a);
gridHelper.position.y = 0.01;
scene.add(gridHelper);

// Ambient particles
const particleCount = 200;
const particleGeom = new THREE.BufferGeometry();
const particlePositions = new Float32Array(particleCount * 3);

for (let i = 0; i < particleCount; i++) {
```

```
            particlePositions[i * 3] = (Math.random() - 0.5) * 40;
            particlePositions[i * 3 + 1] = Math.random() * 20;
            particlePositions[i * 3 + 2] = (Math.random() - 0.5) * 40;
        }

        particleGeom.setAttribute('position', new
THREE.BufferAttribute(particlePositions, 3));

        const particleMat = new THREE.PointsMaterial({
            color: 0x50c878,
            size: 0.05,
            transparent: true,
            opacity: 0.6
        });

        const particles = new THREE.Points(particleGeom, particleMat);
        scene.add(particles);

        // Missile trail particles
        const trailParticles = [];

        function createTrailParticle(position) {
            const particle = new THREE.Mesh(
                new THREE.SphereGeometry(0.1, 4, 4),
                new THREE.MeshBasicMaterial({
                    color: 0xff6b35,
                    transparent: true,
                    opacity: 1
                })
            );
            particle.position.copy(position);
            particle.userData = { life: 1, velocity: new THREE.Vector3(
                (Math.random() - 0.5) * 0.1,
                Math.random() * 0.1,
                (Math.random() - 0.5) * 0.1
            )};
            scene.add(particle);
            trailParticles.push(particle);
        }

        // Launch missile function
        function launchMissile() {
            if (missileCount <= 0) return;

            const availableMissiles = missiles.filter(m => !
m.userData.launched);
            if (availableMissiles.length === 0) return;

            const missile = availableMissiles[0];
            missile.userData.launched = true;
            missile.userData.speed = 0.3;

            // Get world position for trail start
            const worldPos = new THREE.Vector3();
            missile.getWorldPosition(worldPos);
            missile.userData.startPos = worldPos.clone();

            missileCount--;
            launchCount++;
            document.getElementById('missileCount').textContent = missileCount;
            document.getElementById('launchCount').textContent = launchCount;
        }

        // Update launched missiles
```

```
function updateMissiles() {
    missiles.forEach(missile => {
        if (missile.userData.launched) {
            missile.position.x += missile.userData.speed;
            missile.userData.speed *= 1.02;

            // Create trail
            if (Math.random() > 0.5) {
                const worldPos = new THREE.Vector3();
                missile.getWorldPosition(worldPos);
                createTrailParticle(worldPos);
            }

            // Remove when far
            if (missile.position.x > 50) {
                missile.visible = false;
            }
        }
    });

    // Update trail particles
    for (let i = trailParticles.length - 1; i >= 0; i--) {
        const p = trailParticles[i];
        p.userData.life -= 0.02;
        p.material.opacity = p.userData.life;
        p.position.add(p.userData.velocity);
        p.scale.multiplyScalar(0.98);

        if (p.userData.life <= 0) {
            scene.remove(p);
            trailParticles.splice(i, 1);
        }
    }
}

// Raycaster for interaction
const raycaster = new THREE.Raycaster();
const mouse = new THREE.Vector2();
let hoveredObject = null;
let originalMaterial = null;

function onMouseMove(event) {
    mouse.x = (event.clientX / window.innerWidth) * 2 - 1;
    mouse.y = -(event.clientY / window.innerHeight) * 2 + 1;
}

function checkIntersection() {
    raycaster.setFromCamera(mouse, camera);
    const intersects = raycaster.intersectObjects(chassisGroup.children,
true);

    if (hoveredObject && originalMaterial) {
        hoveredObject.material = originalMaterial;
        hoveredObject = null;
        originalMaterial = null;
    }

    if (intersects.length > 0) {
        const obj = intersects[0].object;
        if (obj.material && obj.userData.part) {
            originalMaterial = obj.material;
            hoveredObject = obj;
            obj.material = materials.highlight;
        }
```

```
        }
    }

    // Event listeners
    document.addEventListener('mousemove', onMouseMove);

    document.getElementById('launchBtn').addEventListener('click', () => {
        for (let i = 0; i < 4; i++) {
            setTimeout(() => launchMissile(), i * 200);
        }
    });

    document.getElementById('rotateTurretBtn').addEventListener('click', ()
=> {
        turretBaseGroup.rotation.y += Math.PI / 4;
    });

    document.getElementById('resetBtn').addEventListener('click', () => {
        // Reset missiles
        missiles.forEach(missile => {
            missile.userData.launched = false;
            missile.userData.speed = 0;
            missile.position.set(0.5, 0, missile.position.z);
            missile.visible = true;
        });
        missileCount = 8;
        launchCount = 0;
        document.getElementById('missileCount').textContent = missileCount;
        document.getElementById('launchCount').textContent = launchCount;

        // Reset turret
        turretBaseGroup.rotation.y = 0;

        // Reset camera
        camera.position.set(12, 8, 12);
        controls.target.set(0, 2, 0);
    });

    let autoRotate = true;
    document.getElementById('autoRotateBtn').addEventListener('click', (e)
=> {
        autoRotate = !autoRotate;
        controls.autoRotate = autoRotate;
        e.target.textContent = `Auto Rotate: ${autoRotate ? 'ON' : 'OFF'}`;
    });

    // Window resize
    window.addEventListener('resize', () => {
        camera.aspect = window.innerWidth / window.innerHeight;
        camera.updateProjectionMatrix();
        renderer.setSize(window.innerWidth, window.innerHeight);
    });

    // Animation loop
    const clock = new THREE.Clock();

    function animate() {
        requestAnimationFrame(animate);

        const time = clock.getElapsedTime();

        // Rotate radar
        radarGroup.rotation.y = time * 2;
```

```
            // Animate ambient particles
            const positions = particles.geometry.attributes.position.array;
            for (let i = 0; i < particleCount; i++) {
                positions[i * 3 + 1] += 0.01;
                if (positions[i * 3 + 1] > 20) {
                    positions[i * 3 + 1] = 0;
                }
            }
            particles.geometry.attributes.position.needsUpdate = true;

            // Subtle vehicle movement
            chassisGroup.position.y = Math.sin(time * 2) * 0.02;

            // Update missiles
            updateMissiles();

            // Check hover
            checkIntersection();

            // Update controls
            controls.target.lerp(new THREE.Vector3(0, 2, 0), 0.05);
            controls.update();

            renderer.render(scene, camera);
        }

        animate();
    </script>
</body>
</html>
```

```
#with value
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>3D Missile Launcher</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
```

```
        }
        body {
            overflow: hidden;
            background: #0a0a0f;
            font-family: 'Segoe UI', system-ui, sans-serif;
        }
        #canvas-container {
            width: 100vw;
            height: 100vh;
        }
        #ui-panel {
            position: fixed;
            top: 20px;
            left: 20px;
            background: linear-gradient(135deg, rgba(20, 25, 30, 0.95), rgba(10,
15, 20, 0.98));
            border: 1px solid rgba(80, 200, 120, 0.3);
            border-radius: 12px;
            padding: 20px;
            color: #e0e0e0;
            z-index: 100;
            box-shadow: 0 8px 32px rgba(0, 0, 0, 0.5), inset 0 1px 0
rgba(255,255,255,0.05);
        }
        #ui-panel h1 {
            font-size: 18px;
            margin-bottom: 15px;
            color: #50c878;
            text-transform: uppercase;
            letter-spacing: 2px;
            font-weight: 600;
        }
        .btn {
            display: block;
            width: 100%;
            padding: 12px 20px;
            margin: 8px 0;
            background: linear-gradient(135deg, #1a2f1a, #0d1f0d);
            border: 1px solid #50c878;
            color: #50c878;
            font-size: 13px;
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 1px;
            cursor: pointer;
            border-radius: 6px;
            transition: all 0.3s ease;
        }
        .btn:hover {
            background: linear-gradient(135deg, #50c878, #3da85c);
            color: #0a0f0a;
            box-shadow: 0 0 20px rgba(80, 200, 120, 0.4);
            transform: translateY(-2px);
        }
        .btn:active {
            transform: translateY(0);
        }
        .btn.danger {
            border-color: #ff6b35;
            color: #ff6b35;
            background: linear-gradient(135deg, #2f1a1a, #1f0d0d);
        }
        .btn.danger:hover {
            background: linear-gradient(135deg, #ff6b35, #cc5429);
```

```
            color: #0a0f0a;
            box-shadow: 0 0 20px rgba(255, 107, 53, 0.4);
        }
        .info {
            margin-top: 15px;
            padding-top: 15px;
            border-top: 1px solid rgba(80, 200, 120, 0.2);
            font-size: 11px;
            color: #888;
            line-height: 1.6;
        }
        .stats {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 10px;
            margin-top: 15px;
        }
        .stat-item {
            background: rgba(0,0,0,0.3);
            padding: 10px;
            border-radius: 6px;
            text-align: center;
        }
        .stat-value {
            font-size: 20px;
            font-weight: bold;
            color: #50c878;
        }
        .stat-label {
            font-size: 10px;
            color: #666;
            text-transform: uppercase;
        }
        /* New Degree Display CSS */
        #degree-display {
            position: fixed;
            bottom: 30px;
            right: 30px;
            background: rgba(10, 15, 20, 0.8);
            border-right: 4px solid #50c878;
            padding: 15px 25px;
            color: #50c878;
            font-family: 'Courier New', Courier, monospace;
            z-index: 100;
            pointer-events: none;
            box-shadow: 0 0 15px rgba(0,0,0,0.5);
        }
        .deg-val {
            font-size: 28px;
            font-weight: bold;
        }
        .deg-label {
            font-size: 10px;
            display: block;
            opacity: 0.7;
            letter-spacing: 1px;
        }
    </style>
</head>
<body>
    <div id="canvas-container"></div>

    <div id="ui-panel">
        <h1>Missile Launcher</h1>
```

```
        <button class="btn danger" id="launchBtn">Launch Missile</button>
        <button class="btn" id="rotateTurretBtn">Rotate Turret</button>
        <button class="btn" id="resetBtn">Reset Position</button>
        <button class="btn" id="autoRotateBtn">Auto Rotate: ON</button>
        <div class="stats">
            <div class="stat-item">
                <div class="stat-value" id="missileCount">8</div>
                <div class="stat-label">Missiles</div>
            </div>
            <div class="stat-item">
                <div class="stat-value" id="launchCount">0</div>
                <div class="stat-label">Launched</div>
            </div>
        </div>
        <div class="info">
            Drag to rotate view<br>
            Scroll to zoom<br>
            Click vehicle parts to highlight
        </div>
    </div>

    <div id="degree-display">
        <span class="deg-label">VIEW ANGLE</span>
        <span class="deg-val" id="degVal">0Â°</span>
    </div>

    <script type="importmap">
    {
        "imports": {
            "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
            "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
        }
    }
    </script>

    <script type="module">
        import * as THREE from 'three';
        import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

        // Scene setup
        const container = document.getElementById('canvas-container');
        const scene = new THREE.Scene();

        // Gradient background
        const bgCanvas = document.createElement('canvas');
        bgCanvas.width = 2;
        bgCanvas.height = 512;
        const bgCtx = bgCanvas.getContext('2d');
        const gradient = bgCtx.createLinearGradient(0, 0, 0, 512);
        gradient.addColorStop(0, '#0d1a0d');
        gradient.addColorStop(0.5, '#0a1210');
        gradient.addColorStop(1, '#050a08');
        bgCtx.fillStyle = gradient;
        bgCtx.fillRect(0, 0, 2, 512);
        const bgTexture = new THREE.CanvasTexture(bgCanvas);
        scene.background = bgTexture;

        // Camera
        const camera = new THREE.PerspectiveCamera(60, window.innerWidth /
window.innerHeight, 0.1, 1000);
        camera.position.set(12, 8, 12);

        // Renderer
        const renderer = new THREE.WebGLRenderer({ antialias: true });
```

```
        renderer.setSize(window.innerWidth, window.innerHeight);
        renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
        renderer.shadowMap.enabled = true;
        renderer.shadowMap.type = THREE.PCFSoftShadowMap;
        container.appendChild(renderer.domElement);

        // Controls
        const controls = new OrbitControls(camera, renderer.domElement);
        controls.enableDamping = true;
        controls.dampingFactor = 0.05;
        controls.autoRotate = true;
        controls.autoRotateSpeed = 0.5;
        controls.maxPolarAngle = Math.PI / 2.1;
        controls.minDistance = 8;
        controls.maxDistance = 30;

        // Lighting
        const ambientLight = new THREE.AmbientLight(0x404050, 0.5);
        scene.add(ambientLight);

        const mainLight = new THREE.DirectionalLight(0xfff5e0, 1.2);
        mainLight.position.set(10, 20, 10);
        mainLight.castShadow = true;
        mainLight.shadow.mapSize.width = 2048;
        mainLight.shadow.mapSize.height = 2048;
        mainLight.shadow.camera.near = 0.5;
        mainLight.shadow.camera.far = 50;
        mainLight.shadow.camera.left = -20;
        mainLight.shadow.camera.right = 20;
        mainLight.shadow.camera.top = 20;
        mainLight.shadow.camera.bottom = -20;
        scene.add(mainLight);

        const fillLight = new THREE.DirectionalLight(0x50c878, 0.3);
        fillLight.position.set(-10, 5, -10);
        scene.add(fillLight);

        const rimLight = new THREE.DirectionalLight(0xff6b35, 0.2);
        rimLight.position.set(0, 10, -15);
        scene.add(rimLight);

        // Materials
        const materials = {
            body: new THREE.MeshStandardMaterial({ color: 0x2d3d2d, roughness:
0.7, metalness: 0.3 }),
            dark: new THREE.MeshStandardMaterial({ color: 0x1a251a, roughness:
0.8, metalness: 0.2 }),
            metal: new THREE.MeshStandardMaterial({ color: 0x4a5a4a, roughness:
0.4, metalness: 0.7 }),
            missile: new THREE.MeshStandardMaterial({ color: 0x3a4a3a,
roughness: 0.5, metalness: 0.6 }),
            missileTip: new THREE.MeshStandardMaterial({ color: 0xff6b35,
roughness: 0.3, metalness: 0.5, emissive: 0xff6b35, emissiveIntensity: 0.2 }),
            glass: new THREE.MeshStandardMaterial({ color: 0x88ccaa, roughness:
0.1, metalness: 0.9, transparent: true, opacity: 0.6 }),
            wheel: new THREE.MeshStandardMaterial({ color: 0x1a1a1a, roughness:
0.9, metalness: 0.1 }),
            highlight: new THREE.MeshStandardMaterial({ color: 0x50c878,
roughness: 0.3, metalness: 0.5, emissive: 0x50c878, emissiveIntensity: 0.5 })
        };

        // Missile Launcher Group
        const missileLauncher = new THREE.Group();
        scene.add(missileLauncher);
```

```
        // Chassis
        const chassisGroup = new THREE.Group();
        missileLauncher.add(chassisGroup);

        const body = new THREE.Mesh(new THREE.BoxGeometry(6, 1.5, 3),
materials.body);
        body.position.y = 1.2;
        body.castShadow = true;
        body.receiveShadow = true;
        body.userData = { part: 'body' };
        chassisGroup.add(body);

        const cabin = new THREE.Mesh(new THREE.BoxGeometry(2, 1.8, 2.6),
materials.body);
        cabin.position.set(2.5, 2.3, 0);
        cabin.castShadow = true;
        cabin.userData = { part: 'cabin' };
        chassisGroup.add(cabin);

        const windowFront = new THREE.Mesh(new THREE.BoxGeometry(0.1, 0.8, 2),
materials.glass);
        windowFront.position.set(3.51, 2.5, 0);
        chassisGroup.add(windowFront);

        const windowSide1 = new THREE.Mesh(new THREE.BoxGeometry(1, 0.6, 0.1),
materials.glass);
        windowSide1.position.set(2.8, 2.7, 1.31);
        chassisGroup.add(windowSide1);

        const windowSide2 = windowSide1.clone();
        windowSide2.position.z = -1.31;
        chassisGroup.add(windowSide2);

        const rear = new THREE.Mesh(new THREE.BoxGeometry(2.5, 2.2, 2.8),
materials.dark);
        rear.position.set(-1.8, 2.5, 0);
        rear.castShadow = true;
        rear.userData = { part: 'rear' };
        chassisGroup.add(rear);

        const turretBaseGroup = new THREE.Group();
        turretBaseGroup.position.set(-1.5, 3.6, 0);
        chassisGroup.add(turretBaseGroup);

        const turretBase = new THREE.Mesh(new THREE.CylinderGeometry(1.2, 1.4,
0.6, 8), materials.metal);
        turretBase.castShadow = true;
        turretBase.userData = { part: 'turret' };
        turretBaseGroup.add(turretBase);

        const turretGroup = new THREE.Group();
        turretBaseGroup.add(turretGroup);

        const turret = new THREE.Mesh(new THREE.BoxGeometry(2.5, 0.8, 2),
materials.metal);
        turret.position.y = 0.7;
        turret.castShadow = true;
        turretGroup.add(turret);

        const missilePodGroup = new THREE.Group();
        missilePodGroup.position.set(-0.8, 1.3, 0);
        missilePodGroup.rotation.z = Math.PI * 0.1;
        turretGroup.add(missilePodGroup);
```

```
        function createMissilePod(zOffset) {
            const podGroup = new THREE.Group();
            podGroup.position.z = zOffset;
            const podFrame = new THREE.Mesh(new THREE.BoxGeometry(2.8, 1.4,
0.6), materials.dark);
            podFrame.castShadow = true;
            podGroup.add(podFrame);
            return podGroup;
        }

        const leftPod = createMissilePod(0.7);
        const rightPod = createMissilePod(-0.7);
        missilePodGroup.add(leftPod);
        missilePodGroup.add(rightPod);

        const missiles = [];
        let missileCount = 8;
        let launchCount = 0;

        function createMissile(zPos, pod) {
            const missileGroup = new THREE.Group();
            missileGroup.position.set(0.5, 0, zPos);
            const mBody = new THREE.Mesh(new THREE.CylinderGeometry(0.12, 0.12,
1.2, 8), materials.missile);
            mBody.rotation.z = Math.PI / 2;
            mBody.castShadow = true;
            missileGroup.add(mBody);
            const tip = new THREE.Mesh(new THREE.ConeGeometry(0.12, 0.3, 8),
materials.missileTip);
            tip.rotation.z = -Math.PI / 2;
            tip.position.x = 0.75;
            missileGroup.add(tip);
            missileGroup.userData = { launched: false, speed: 0 };
            pod.add(missileGroup);
            missiles.push(missileGroup);
        }

        createMissile(0.15, leftPod); createMissile(-0.15, leftPod);
        createMissile(0.15, rightPod); createMissile(-0.15, rightPod);

        function createWheel(x, z) {
            const wheelGroup = new THREE.Group();
            wheelGroup.position.set(x, 0.5, z);
            const wheel = new THREE.Mesh(new THREE.CylinderGeometry(0.5, 0.5,
0.4, 16), materials.wheel);
            wheel.rotation.x = Math.PI / 2;
            wheel.castShadow = true;
            wheelGroup.add(wheel);
            return wheelGroup;
        }

        [[2, 1.7], [2, -1.7], [0.5, 1.7], [0.5, -1.7], [-1, 1.7], [-1, -1.7],
[-2.5, 1.7], [-2.5, -1.7]].forEach(pos => {
            chassisGroup.add(createWheel(pos[0], pos[1]));
        });

        const radarGroup = new THREE.Group();
        radarGroup.position.set(3, 3.5, 0);
        chassisGroup.add(radarGroup);
        radarGroup.add(new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.15, 0.5,
6), materials.metal));
        const dish = new THREE.Mesh(new THREE.SphereGeometry(0.3, 8, 4, 0,
Math.PI * 2, 0, Math.PI / 2), materials.metal);
```

```
        dish.position.y = 0.85; dish.rotation.x = Math.PI / 4;
        radarGroup.add(dish);

        scene.add(new THREE.GridHelper(50, 50, 0x2a3a2a, 0x1a251a));

        const trailParticles = [];
        function createTrailParticle(position) {
            const particle = new THREE.Mesh(new THREE.SphereGeometry(0.1, 4, 4),
new THREE.MeshBasicMaterial({ color: 0xff6b35, transparent: true, opacity:
1 }));
            particle.position.copy(position);
            particle.userData = { life: 1, velocity: new
THREE.Vector3((Math.random()-0.5)*0.1, Math.random()*0.1,
(Math.random()-0.5)*0.1)};
            scene.add(particle);
            trailParticles.push(particle);
        }

        function launchMissile() {
            const available = missiles.filter(m => !m.userData.launched);
            if (available.length === 0 || missileCount <= 0) return;
            const m = available[0];
            m.userData.launched = true;
            m.userData.speed = 0.3;
            missileCount--; launchCount++;
            document.getElementById('missileCount').textContent = missileCount;
            document.getElementById('launchCount').textContent = launchCount;
        }

        const raycaster = new THREE.Raycaster();
        const mouse = new THREE.Vector2();
        let hoveredObject = null;
        let originalMaterial = null;

        document.addEventListener('mousemove', (e) => {
            mouse.x = (e.clientX / window.innerWidth) * 2 - 1;
            mouse.y = -(e.clientY / window.innerHeight) * 2 + 1;
        });

        document.getElementById('launchBtn').addEventListener('click', () => {
            for (let i = 0; i < 4; i++) setTimeout(() => launchMissile(), i *
200);
        });

        document.getElementById('rotateTurretBtn').addEventListener('click', ()
=> {
            turretBaseGroup.rotation.y += Math.PI / 4;
        });

        document.getElementById('resetBtn').addEventListener('click', () => {
            missiles.forEach(m => { m.userData.launched = false; m.position.x =
0.5; m.visible = true; });
            missileCount = 8; launchCount = 0;
            document.getElementById('missileCount').textContent = 8;
            document.getElementById('launchCount').textContent = 0;
            turretBaseGroup.rotation.y = 0;
            camera.position.set(12, 8, 12);
        });

        let autoRotate = true;
        document.getElementById('autoRotateBtn').addEventListener('click', (e)
=> {
            autoRotate = !autoRotate;
            controls.autoRotate = autoRotate;
```

```
            e.target.textContent = `Auto Rotate: ${autoRotate ? 'ON' : 'OFF'}`;
        });

        window.addEventListener('resize', () => {
            camera.aspect = window.innerWidth / window.innerHeight;
            camera.updateProjectionMatrix();
            renderer.setSize(window.innerWidth, window.innerHeight);
        });

        const clock = new THREE.Clock();
        const degElement = document.getElementById('degVal');

        function animate() {
            requestAnimationFrame(animate);
            const time = clock.getElapsedTime();
            radarGroup.rotation.y = time * 2;
            chassisGroup.position.y = Math.sin(time * 2) * 0.02;

            // Calculate camera degree (0-360)
            const angle = Math.atan2(camera.position.x, camera.position.z);
            let degrees = Math.round(angle * (180 / Math.PI));
            if (degrees < 0) degrees += 360;
            degElement.textContent = `${degrees}Â°`;

            missiles.forEach(m => {
                if (m.userData.launched) {
                    m.position.x += m.userData.speed;
                    m.userData.speed *= 1.02;
                    if (Math.random() > 0.5) {
                        const wp = new THREE.Vector3(); m.getWorldPosition(wp);
                        createTrailParticle(wp);
                    }
                    if (m.position.x > 50) m.visible = false;
                }
            });

            for (let i = trailParticles.length - 1; i >= 0; i--) {
                const p = trailParticles[i];
                p.userData.life -= 0.02;
                p.material.opacity = p.userData.life;
                p.position.add(p.userData.velocity);
                if (p.userData.life <= 0) { scene.remove(p);
trailParticles.splice(i, 1); }
            }

            controls.update();
            renderer.render(scene, camera);
        }
        animate();
    </script>
</body>
</html>
```